

**SIMATS ENGINEERING**  
**Department of Computer Science & Engineering**  
**ASSESSMENT TOOL 1- EXPERIMENTS**

---

|                       |                                                                                                                                                                                                                                                                                    |                      |                                           |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|-------------------------------------------|
| <b>Course Code:</b>   | <b>CSA1209 &amp; CSA1215</b>                                                                                                                                                                                                                                                       | <b>Course Title:</b> | <b>Computer Architecture</b>              |
| <b>CO Assessed:</b>   | <b>CO3</b>                                                                                                                                                                                                                                                                         | <b>Assessment:</b>   | <b>ASSESSMENT TOOL 1-<br/>EXPERIMENTS</b> |
| <b>Weightage:</b>     | <b>40%</b>                                                                                                                                                                                                                                                                         | <b>Total Marks:</b>  | <b>25</b>                                 |
| <b>CO3 Statement:</b> | <i>Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT). (BL6)</i> |                      |                                           |
| <b>Student Name:</b>  | <b>S.TANISHA</b>                                                                                                                                                                                                                                                                   | <b>Reg.No.:</b>      | <b>192465008</b>                          |
| <b>Department:</b>    | <b>CSE CS</b>                                                                                                                                                                                                                                                                      | <b>Date:</b>         | <b>21.08.26</b>                           |

**ANNEXURE A**

1. Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyze the sequence of operations and the role of the ALU in processing data.
  2. Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyze how multiple bus structures improve parallel data transfer and reduce bottlenecks.
  3. Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare it with a non-pipelined system to evaluate the effectiveness of pipelining.
  4. Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stalling, data forwarding, and branch prediction to resolve these hazards and improve performance.
  5. Analyze a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.
-

**Code of Conduct:**

*I \_\_K Sreelatha(192425248) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.*

**Signature of the student: Sree K**

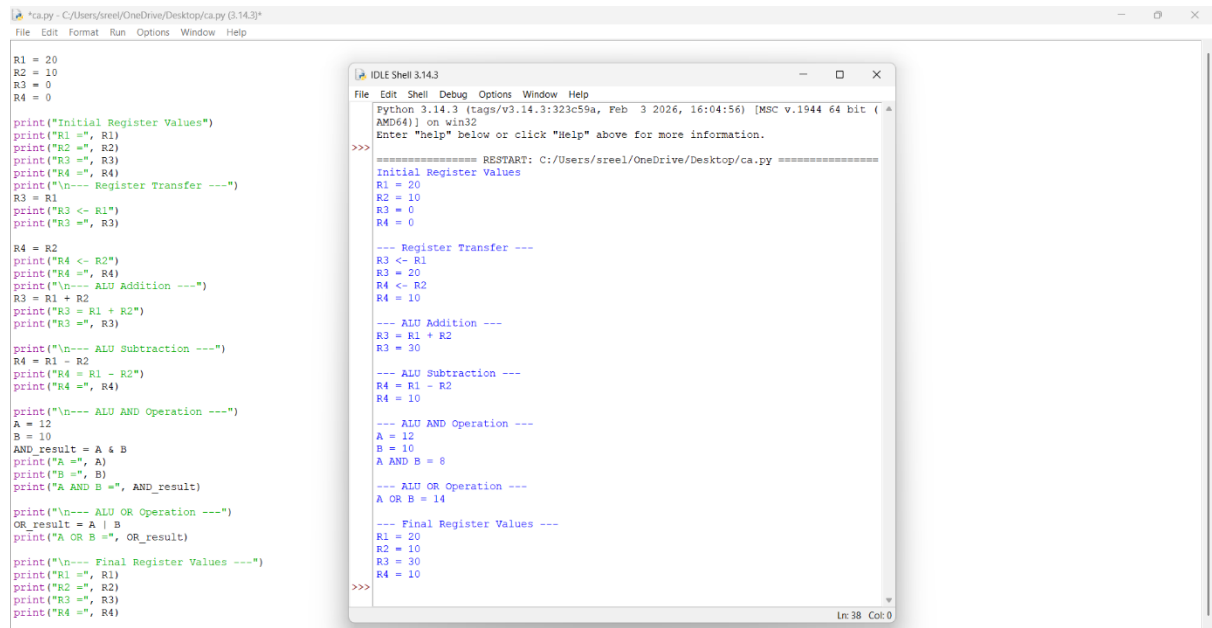
**MARKS ALLOCATION**

|    | Understanding of Concepts (20%) | Experimental Design (20%) | Use of Tools (25%) | Analysis of Results (20%) | Documentation (15%) |
|----|---------------------------------|---------------------------|--------------------|---------------------------|---------------------|
| Q1 |                                 |                           |                    |                           |                     |
| Q2 |                                 |                           |                    |                           |                     |
| Q3 |                                 |                           |                    |                           |                     |
| Q4 |                                 |                           |                    |                           |                     |
| Q5 |                                 |                           |                    |                           |                     |

**RUBRICS FOR CALCULATION:**

| Criteria                  | Weightage | Level 4 – Exemplary                                                                           | Level 3 – Proficient                                        | Level 2 – Developing                                | Level 1 – Beginning                              |
|---------------------------|-----------|-----------------------------------------------------------------------------------------------|-------------------------------------------------------------|-----------------------------------------------------|--------------------------------------------------|
| Understanding of Concepts | 25        | Demonstrates thorough understanding and effectively integrates multiple concepts/technologies | Good understanding with proper integration of most concepts | Basic understanding; limited or partial integration | Poor understanding; unable to integrate concepts |
| Experimental Design       | 30        | Well-planned, systematic implementation with optimal solution                                 | Correct implementation with minor issues                    | Basic implementation with errors or inefficiencies  | Incorrect or incomplete implementation           |
| Use of Tools              | 20        | Effective and advanced use of tools/technologies with proper configuration                    | Appropriate use of tools with correct execution             | Limited or inconsistent use of tools                | Incorrect or minimal use of tools                |
| Analysis of Results       | 15        | Insightful analysis with accurate interpretation and validation of results                    | Good analysis with reasonable interpretation                | Basic analysis with limited insights                | Poor or incorrect analysis of results            |
| Documentation             | 10        | Clear, well-structured documentation with diagrams, code, and results                         | Organized documentation with necessary details              | Basic documentation with missing details            | Poor or incomplete documentation                 |

# 1.Register Transfer + Arithmetic and Logic Operations



The screenshot shows a Python IDE with two windows. The main window displays a Python script for Register Transfer and ALU operations. The script initializes four registers (R1, R2, R3, R4) with values 20, 10, 0, and 0 respectively. It then performs a series of operations: R3 is assigned the value of R1; R4 is assigned the value of R2; R3 is incremented by R2 (R3 = R1 + R2); R4 is decremented by R2 (R4 = R1 - R2); R3 is ANDed with R2 (R3 = R1 & R2); and R4 is ORed with R2 (R4 = R1 | R2). Finally, the values of all four registers are printed.

```
R1 = 20
R2 = 10
R3 = 0
R4 = 0

print("Initial Register Values")
print("R1 =", R1)
print("R2 =", R2)
print("R3 =", R3)
print("R4 =", R4)
print("\n--- Register Transfer ---")
R3 = R1
print("R3 =", R3)

R4 = R2
print("R4 =", R4)
print("\n--- ALU Addition ---")
R3 = R1 + R2
print("R3 =", R3)

print("\n--- ALU Subtraction ---")
R4 = R1 - R2
print("R4 =", R4)

print("\n--- ALU AND Operation ---")
A = 12
B = 10
AND_result = A & B
print("A =", A)
print("B =", B)
print("A AND B =", AND_result)

print("\n--- ALU OR Operation ---")
OR_result = A | B
print("A OR B =", OR_result)

print("\n--- Final Register Values ---")
print("R1 =", R1)
print("R2 =", R2)
print("R3 =", R3)
print("R4 =", R4)
```

The second window shows the output of the program, displaying the initial register values, the results of the register transfer, the ALU operations, and the final register values.

```
Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 16:04:56) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> ===== RESTART: C:/Users/sreel/OneDrive/Desktop/ca.py =====
Initial Register Values
R1 = 20
R2 = 10
R3 = 0
R4 = 0

--- Register Transfer ---
R3 <- R1
R3 = 20
R4 <- R2
R4 = 10

--- ALU Addition ---
R3 = R1 + R2
R3 = 30

--- ALU Subtraction ---
R4 = R1 - R2
R4 = 10

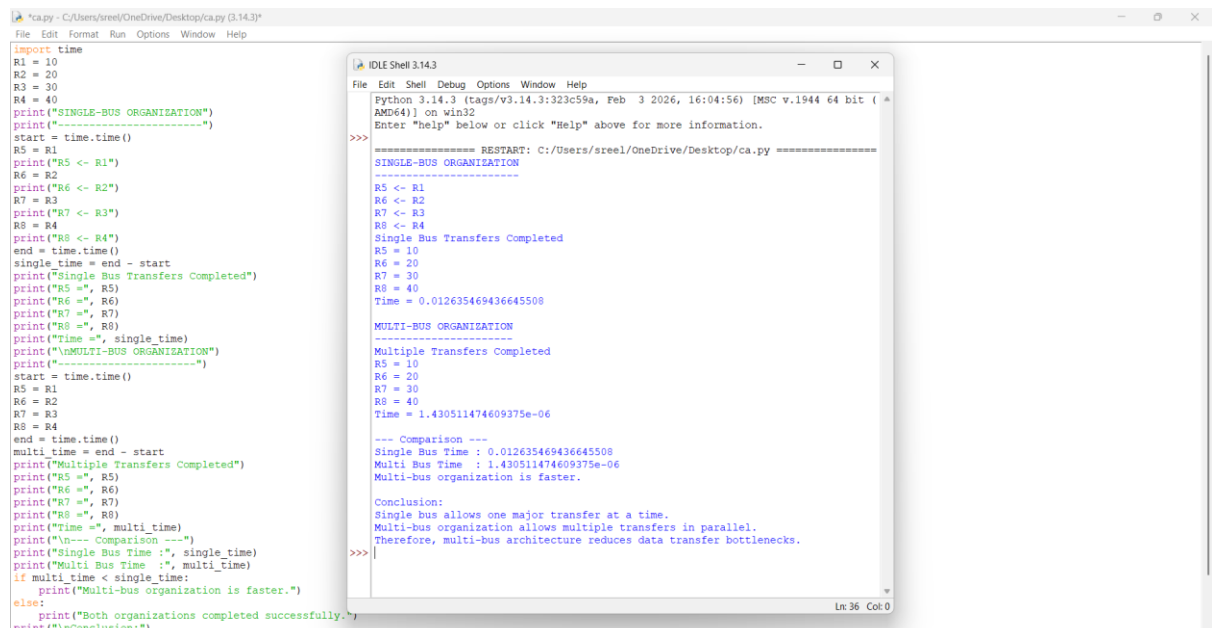
--- ALU AND Operation ---
A = 12
B = 10
A AND B = 8

--- ALU OR Operation ---
A OR B = 14

--- Final Register Values ---
R1 = 20
R2 = 10
R3 = 30
R4 = 10

>>>
```

# 2.Single-Bus and Multi-Bus Organization



The screenshot shows a Python IDE with two windows. The main window displays a Python script for Single-Bus and Multi-Bus Organization. The script initializes eight registers (R1, R2, R3, R4, R5, R6, R7, R8) with values 10, 20, 30, 40, 10, 20, 30, and 40 respectively. It then performs a series of operations: R5 is assigned the value of R1; R6 is assigned the value of R2; R7 is assigned the value of R3; R8 is assigned the value of R4; R5 is incremented by R2 (R5 = R1 + R2); R6 is decremented by R2 (R6 = R1 - R2); R7 is ANDed with R2 (R7 = R1 & R2); and R8 is ORed with R2 (R8 = R1 | R2). Finally, the values of all eight registers are printed.

```
import time
R1 = 10
R2 = 20
R3 = 30
R4 = 40

print("SINGLE-BUS ORGANIZATION")
print("-----")
start = time.time()
R5 = R1
print("R5 <- R1")
R6 = R2
print("R6 <- R2")
R7 = R3
print("R7 <- R3")
R8 = R4
print("R8 <- R4")
end = time.time()
single_time = end - start
print("Single Bus Transfers Completed")
print("R5 =", R5)
print("R6 =", R6)
print("R7 =", R7)
print("R8 =", R8)
print("Time =", single_time)

print("\nMULTI-BUS ORGANIZATION")
print("-----")
start = time.time()
R5 = R1
R6 = R2
R7 = R3
R8 = R4
end = time.time()
multi_time = end - start
print("Multiple Transfers Completed")
print("R5 =", R5)
print("R6 =", R6)
print("R7 =", R7)
print("R8 =", R8)
print("Time =", multi_time)

print("\n--- Comparison ---")
print("Single Bus Time :", single_time)
print("Multi Bus Time :", multi_time)
if multi_time < single_time:
    print("Multi-bus organization is faster.")
else:
    print("Both organizations completed successfully.")
print("\nConclusion:")
```

The second window shows the output of the program, displaying the initial register values, the results of the register transfer, the ALU operations, and the final register values.

```
Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 16:04:56) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> ===== RESTART: C:/Users/sreel/OneDrive/Desktop/ca.py =====
SINGLE-BUS ORGANIZATION
-----
R5 <- R1
R6 <- R2
R7 <- R3
R8 <- R4
Single Bus Transfers Completed
R5 = 10
R6 = 20
R7 = 30
R8 = 40
Time = 0.012635469436645508

MULTI-BUS ORGANIZATION
-----
Multiple Transfers Completed
R5 = 10
R6 = 20
R7 = 30
R8 = 40
Time = 1.430511474609375e-06

--- Comparison ---
Single Bus Time : 0.012635469436645508
Multi Bus Time : 1.430511474609375e-06
Multi-bus organization is faster.

Conclusion:
Single bus allows one major transfer at a time.
Multi-bus organization allows multiple transfers in parallel.
Therefore, multi-bus architecture reduces data transfer bottlenecks.

>>>
```

### 3. Five-Stage Instruction Pipeline

```
ca.py - C:/Users/sreel/OneDrive/Desktop/ca.py (3.14.3)
File Edit Format Run Options Window Help

instructions = [
    "I1",
    "I2",
    "I3",
    "I4",
    "I5"
]

stages = ["IF", "ID", "EX", "MEM", "WB"]
print("5-STAGE PIPELINE")
print("=====")
cycles = len(instructions) + len(stages) - 1
print("\nPipeline Execution Table:\n")
print("Cycle\t", end="")
for i in range(1, len(instructions) + 1):
    print(i, end="\t")
print()
for cycle in range(cycles):
    print(cycle + 1, "\t", end="")
    for instruction in range(len(instructions)):
        stage = cycle - instruction
        if 0 <= stage < len(stages):
            print(stages[stage], end="\t")
        else:
            print("-", end="\t")
    print()

non_pipeline_cycles = len(instructions) * len(stages)
pipeline_cycles = cycles
speedup = non_pipeline_cycles / pipeline_cycles
print("\n--- Performance ---")
print("Number of Instructions : ", len(instructions))
print("Number of Stages : ", len(stages))
print("Non-Pipelined Cycles : ", non_pipeline_cycles)
print("Pipelined Cycles : ", pipeline_cycles)
print("Speedup = ",
      round(speedup, 2))
throughput_non = len(instructions) / non_pipeline_cycles
throughput_pipe = len(instructions) / pipeline_cycles
print("Non-Pipelined Throughput = ",
      round(throughput_non, 3),
      "instructions/cycle")
print("Pipelined Throughput = ",
      round(throughput_pipe, 3),
      "instructions/cycle")
print("\nConclusion:")
print("Pipelining allows multiple instructions to be processed")
print("simultaneously in different stages.")
print("Therefore, pipeline execution improves throughput.")
```

```
IDLE Shell 3.14.3
Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 16:04:56) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/sreel/OneDrive/Desktop/ca.py =====
5-STAGE PIPELINE
=====

Pipeline Execution Table:

Cycle  1    2    3    4    5
1      IF    -    -    -    -
2      ID    IF   -    -    -
3      EX    ID    IF   -    -
4      MEM   EX    ID    IF   -
5      WB    MEM   EX    ID    IF
6      -     WB    MEM   EX    ID
7      -     -     WB    MEM   EX
8      -     -     -     WB    MEM
9      -     -     -     -     WB

--- Performance ---
Number of Instructions : 5
Number of Stages : 5
Non-Pipelined Cycles : 25
Pipelined Cycles : 9
Speedup = 2.78
Non-Pipelined Throughput = 0.2 instructions/cycle
Pipelined Throughput = 0.556 instructions/cycle

Conclusion:
Pipelining allows multiple instructions to be processed
simultaneously in different stages.
Therefore, pipeline execution improves throughput.

>>>
```

### 4.Data, Control and Structural Hazards

```
ca.py - C:/Users/sreel/OneDrive/Desktop/ca.py (3.14.3)
File Edit Format Run Options Window Help

print("PIPELINE HAZARD SIMULATION")
print("=====")

instructions = [
    "I1: ADD R1,R2,R3",
    "I2: SUB R4,R1,R5",
    "I3: LOAD R6,R7",
    "I4: BEQ R6,R0",
    "I5: ADD R8,R9,R10"
]

print("\nInstructions:")
for instruction in instructions:
    print(instruction)

print("\n--- DATA HAZARD ---")
print("I1 produces R1.")
print("I2 requires R1.")
print("Therefore, I2 has a data dependency on I1.")

print("\nWithout Forwarding:")
print("I1 -> EX")
print("I2 -> STALL")
print("I2 waits for R1")

print("\nWith Data Forwarding:")
print("I1 result is forwarded directly to I2.")
print("Stall cycles are reduced.")

print("\n--- CONTROL HAZARD ---")
print("I4 is a branch instruction.")
print("The processor may not know the next instruction immediately.")

print("\nWithout Branch Prediction:")
print("Pipeline waits until branch decision is known.")

print("\nWith Branch Prediction:")
print("Processor predicts the branch direction.")
print("Instructions continue executing speculatively.")
```

```
IDLE Shell 3.14.3
Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 16:04:56) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/sreel/OneDrive/Desktop/ca.py =====
PIPELINE HAZARD SIMULATION
=====

Instructions:
I1: ADD R1,R2,R3
I2: SUB R4,R1,R5
I3: LOAD R6,R7
I4: BEQ R6,R0
I5: ADD R8,R9,R10

--- DATA HAZARD ---
I1 produces R1.
I2 requires R1.
Therefore, I2 has a data dependency on I1.

Without Forwarding:
I1 -> EX
I2 -> STALL
I2 waits for R1

With Data Forwarding:
I1 result is forwarded directly to I2.
Stall cycles are reduced.

--- CONTROL HAZARD ---
I4 is a branch instruction.
The processor may not know the next instruction immediately.

Without Branch Prediction:
Pipeline waits until branch decision is known.

With Branch Prediction:
Processor predicts the branch direction.
Instructions continue executing speculatively.

>>>
```

```

File Edit Format Run Options Window Help
print("The processor may not know the next instruction immediately.")

print("\nWithout Branch Prediction:")
print("Pipeline waits until branch decision is known.")

print("\nWith Branch Prediction:")
print("Processor predicts the branch direction.")
print("Instructions continue executing speculatively.")

print("\n--- STRUCTURAL HAZARD ---")

print("Assume instruction fetch and data memory use")
print("the same hardware resource.")

print("If both require the resource at the same time,")
print("a structural hazard occurs.")

print("\nSolution:")
print("Use separate instruction and data resources.")
print("This allows both operations to occur without conflict.")

print("\n--- PERFORMANCE COMPARISON ---")

cycles_without = 15
stall_cycles = 3
cycles_with = cycles_without - stall_cycles

print("Cycles without hazard handling =", cycles_without)
print("Stall cycles      =", stall_cycles)
print("Cycles after optimization      =", cycles_with)

improvement = ((cycles_without - cycles_with)
               / cycles_without) * 100

print("Performance improvement =",
      round(improvement, 2), "%")

print("\nConclusion:")
print("Data forwarding reduces data stalls.")
print("Branch prediction reduces control-hazard penalties.")
print("Separate resources reduce structural hazards.")

```

```

IDLE Shell 3.14.3
File Edit Shell Debug Options Window Help

I1 -> EX
I2 -> STALL
I2 waits for R1

With Data Forwarding:
I1 result is forwarded directly to I2.
Stall cycles are reduced.

--- CONTROL HAZARD ---
I4 is a branch instruction.
The processor may not know the next instruction immediately.

Without Branch Prediction:
Pipeline waits until branch decision is known.

With Branch Prediction:
Processor predicts the branch direction.
Instructions continue executing speculatively.

--- STRUCTURAL HAZARD ---
Assume instruction fetch and data memory use
the same hardware resource.
If both require the resource at the same time,
a structural hazard occurs.

Solution:
Use separate instruction and data resources.
This allows both operations to occur without conflict.

--- PERFORMANCE COMPARISON ---
Cycles without hazard handling = 15
Stall cycles      = 3
Cycles after optimization      = 12
Performance improvement = 20.0 %

Conclusion:
Data forwarding reduces data stalls.
Branch prediction reduces control-hazard penalties.
Separate resources reduce structural hazards.
>>>

```

## 5. Superscalar, Out-of-Order, Speculative Execution and SMT

```

ca.py - C:/Users/sreel/OneDrive/Desktop/ca.py (3.14.3)
File Edit Format Run Options Window Help

instructions = [
    ("I1", 3),
    ("I2", 1),
    ("I3", 2),
    ("I4", 1),
    ("I5", 3),
    ("I6", 2)
]

print("SUPERSCALAR PROCESSOR SIMULATION")
print("=====")

print("\nInstructions and Execution Time:")
for instruction, time_required in instructions:
    print(instruction, "->", time_required, "cycles")

single_cycles = 0

for instruction, time_required in instructions:
    single_cycles += time_required

print("\n--- Single Issue Processor ---")
print("Total cycles =", single_cycles)

print("\n--- Two-Way Superscalar ---")

cycle = 1
index = 0

while index < len(instructions):
    print("Cycle", cycle, ".", end=" ")

    count = 0

    while count < 2 and index < len(instructions):
        print(instructions[index][0], end=" ")
        index += 1
        count += 1

    cycle += 1

print("\n--- Out-of-Order Execution ---")
print("Execution order based on availability:")
print("I2 (1 cycle)")
print("I4 (1 cycle)")
print("I3 (2 cycle)")
print("I6 (2 cycle)")
print("I1 (3 cycle)")
print("I5 (3 cycle)")

print("\n--- Speculative Execution ---")
print("Branch prediction = TAKEN")
print("Processor executes predicted instructions.")

print("\n--- SMT / Hyper-Threading ---")
print("Thread 1: ['I1-I1', 'I1-I2', 'I1-I3']")

```

```

IDLE Shell 3.14.3
File Edit Shell Debug Options Window Help

Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 16:04:56) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>

===== RESTART: C:/Users/sreel/OneDrive/Desktop/ca.py =====
SUPERSCALAR PROCESSOR SIMULATION
=====

Instructions and Execution Time:
I1 -> 3 cycles
I2 -> 1 cycles
I3 -> 2 cycles
I4 -> 1 cycles
I5 -> 3 cycles
I6 -> 2 cycles

--- Single Issue Processor ---
Total cycles = 12

--- Two-Way Superscalar ---
Cycle 1 : I1 I2
Cycle 2 : I3 I4
Cycle 3 : I5 I6
Superscalar cycles = 3

--- Out-of-Order Execution ---
Execution order based on availability:
I2 (1 cycle)
I4 (1 cycle)
I3 (2 cycle)
I6 (2 cycle)
I1 (3 cycle)
I5 (3 cycle)

--- Speculative Execution ---
Branch prediction = TAKEN
Processor executes predicted instructions.

--- SMT / Hyper-Threading ---
Thread 1: ['I1-I1', 'I1-I2', 'I1-I3']

```

```
ca.py - C:/Users/sreel/OneDrive/Desktop/ca.py (3.14.3)
File Edit Format Run Options Window Help

branch_prediction = True

if branch_prediction:
    print("Branch prediction = TAKEN")
    print("Processor executes predicted instructions.")
else:
    print("Branch prediction = NOT TAKEN")

print("\n--- SMT / Hyper-Threading ---")

thread1 = ["T1-I1", "T1-I2", "T1-I3"]
thread2 = ["T2-I1", "T2-I2", "T2-I3"]

print("Thread 1:", thread1)
print("Thread 2:", thread2)

print("\nParallel execution:")

for i in range(3):
    print("Cycle", i + 1, ":", thread1[i], thread2[i])

print("\n--- PERFORMANCE COMPARISON ---")

speedup = single_cycles / superscalar_cycles

print("Single Issue Cycles :", single_cycles)
print("Superscalar Cycles :", superscalar_cycles)
print("Speedup : ", round(speedup, 2), "x")

utilization = 2 / 2 * 100

print("Superscalar Resource Utilization =",
      utilization, "%")

print("\nConclusion:")
print("Superscalar processors issue multiple instructions")
print("Out-of-order execution uses available resources efficiently")
print("Speculative execution improves performance around branches")
print("SMT allows multiple threads to share processor resources")

IDLE Shell 3.14.3
File Edit Shell Debug Options Window Help

--- Two-Way Superscalar ---
Cycle 1 : I1 I2
Cycle 2 : I3 I4
Cycle 3 : I5 I6
Superscalar cycles = 3

--- Out-of-Order Execution ---
Execution order based on availability:
I2 ( 1 cycle )
I4 ( 1 cycle )
I3 ( 2 cycle )
I6 ( 2 cycle )
I1 ( 3 cycle )
I5 ( 3 cycle )

--- Speculative Execution ---
Branch prediction = TAKEN
Processor executes predicted instructions.

--- SMT / Hyper-Threading ---
Thread 1: ['T1-I1', 'T1-I2', 'T1-I3']
Thread 2: ['T2-I1', 'T2-I2', 'T2-I3']

Parallel execution:
Cycle 1 : T1-I1 + T2-I1
Cycle 2 : T1-I2 + T2-I2
Cycle 3 : T1-I3 + T2-I3

--- PERFORMANCE COMPARISON ---
Single Issue Cycles : 12
Superscalar Cycles : 3
Speedup : 4.0 x
Superscalar Resource Utilization = 100.0 %

Conclusion:
Superscalar processors issue multiple instructions per cycle.
Out-of-order execution uses available resources efficiently.
Speculative execution improves performance around branches.
SMT allows multiple threads to share processor resources.

Ln:58 Col:0
```